

06 — Obfuscation & AV Evasion

The Problem

Anti-virus software uses several strategies to detect malicious tools:

Strategy 1: Hash-Based Signatures

AV vendors maintain databases containing the SHA-256 (and MD5, SHA-1) hashes of known-bad files. When you try to execute a file, Windows Defender computes its hash and checks against the database. If it matches, execution is blocked.

The weakness: This only works for files that have been seen before in exactly the same byte-for-byte form. Any change — even flipping a single bit — produces a completely different hash.

Strategy 2: Static String Scanning

AV parsers scan binary files looking for suspicious text strings:

- API function names: `VirtualAllocEx`, `WriteProcessMemory`, `CreateRemoteThread`
- System paths: `C:\Windows\System32\`, `HKEY_LOCAL_MACHINE\`
- Command strings: `cmd.exe /c`, `powershell -enc`
- Tool signatures: `Mimikatz`, `Metasploit`, `Cobalt Strike`

Any readable string in a compiled binary is a potential detection signature.

Strategy 3: Script Language Detection

AV engines include parsers for Python, PowerShell, VBScript, JavaScript, and other common scripting languages. If a script file fails to parse as any known language, the AV cannot reason about it and may apply heuristic scanning instead of signature matching.

How JOCKY Defeats Each Strategy

AV Strategy	JOCKY's Defence
Hash-based	Polymorphic compilation: three different sources of entropy inject different bytes every build → different SHA-256 every time
String scanning	XOR encryption: all string literals are encrypted in the binary → no readable text
Script parsing	Unknown syntax: <code>.jk</code> files use unique keywords (<code>check</code> , <code>loop</code> , <code>give</code> , <code>func</code> , <code>also</code>) that parse correctly only by the JOCKY compiler

Obfuscation Architecture

The `ObfuscationPasses` class wraps the `ir.Module` (the LLVM IR module) and applies three passes:

```
class ObfuscationPasses:
    def __init__(self, module: ir.Module):
        self.module = module

    def run_all(self) -> ir.Module:
        self._pass_polymorphic_marker()    # Pass 1
        self._pass_string_encryption()      # Pass 2
        self._pass_instruction_substitution() # Pass 3
        return self.module
```

All three passes mutate the IR module in-place. The modified module is then compiled to a `.o` file by LLVM. The mutations change the bytes in the `.o` and `.exe` without changing the program's behaviour (except for Pass 2, which encrypts strings — currently the native binary does not decrypt at runtime).

Obfuscation is skipped for JIT mode because the JIT mode's Python callbacks receive pointers to the IR's string globals. Encrypted string bytes passed to `report()` would print as garbage. The obfuscation is only meaningful for binary output — the demos show encrypted strings in the `.ll` IR file and different SHA-256 hashes on every compile.

Pass 1 — Polymorphic Build-ID

Goal

Ensure every compilation produces a binary with a different SHA-256 hash — even when the source code is identical.

Implementation

```
def _pass_polymorphic_marker(self) -> None:
    i8      = ir.IntType(8)
    arr16   = ir.ArrayType(i8, 16)
    marker  = ir.GlobalVariable(self.module, arr16, name='_jocky_build_id')
    marker.global_constant = True
    marker.linkage          = 'internal'
    build_id = bytearray(random.randint(0, 255) for _ in range(16))
    marker.initializer = ir.Constant(arr16, build_id)
```

What this creates in the IR:

```
@_jocky_build_id = internal constant [16 x i8]
c"\a3\xf1\2c\7b\d9\0e\44\88\xf3\1a\b7\c2\55\92\6d\xf0"
```

Why 16 bytes? It is enough to make every binary uniquely identifiable (2^{128} possible values) while being small enough to add no noticeable overhead.

Why `internal`? The symbol is not exported — it does not appear in the binary's export table where tools might notice it and identify it as an obfuscation marker.

Why `constant`? Read-only data. It goes into the `.rdata` section (Windows) or `.rodata` (Linux), not the writable `.data` section.

Proof

Running the compiler twice on the same source:

```
Build 1: SHA-256 =
1e3b50241d5d98ad963f6c8bf88d6cff5d52c22dbfeb7a53dade02b11f0fce0d
Build 2: SHA-256 =
1304a8d2aa93f41bda1823727f85bf1c72b05f319ed8240a88e88957a428bfcf
```

The hashes differ entirely because 16 different random bytes are embedded at a random offset in the data section.

Pass 2 — String XOR Encryption

Goal

Eliminate all readable text from the compiled binary so that static string scanners find no suspicious content.

The XOR cipher

XOR (exclusive-or) is a bitwise operation:

- `0 XOR 0 = 0`
- `1 XOR 0 = 1`
- `0 XOR 1 = 1`
- `1 XOR 1 = 0`

Key properties that make it useful for encryption:

- `plaintext XOR key = ciphertext`
- `ciphertext XOR key = plaintext` (XOR is its own inverse)
- Fast (one CPU instruction per byte)

For each string, we pick a random key byte (1–255) and XOR every byte of the string with it:

plaintext:	H	e	l	l	o	\0
ASCII:	72	101	108	108	111	0
key:	74	74	74	74	74	74
XOR:	2	39	38	38	37	74

The null terminator (`\0`) is also XOR'd (key is never 0, so `\0 XOR key = key ≠ 0`). This eliminates the easily-detectable null terminator from the binary.

Implementation

```
def _pass_string_encryption(self) -> None:
    # Collect all string globals
    string_globals = [
        gv for gv in self.module.global_values
        if (isinstance(gv, ir.GlobalVariable)
            and gv.initializer is not None
            and isinstance(gv.type.pointee, ir.ArrayType)
            and isinstance(gv.type.pointee.element, ir.IntType)
            and gv.type.pointee.element.width == 8
            and gv.name.startswith('.jk_str'))
    ]

    for gv in string_globals:
        raw = bytes(gv.initializer.constant)
        key = random.randint(1, 255)          # fresh key per string per build
        encrypted = bytearray(b ^ key for b in raw)
        gv.initializer = ir.Constant(gv.type.pointee, encrypted)

        # Store the key in a companion global
        key_arr = ir.ArrayType(ir.IntType(8), 1)
        key_gv = ir.GlobalVariable(self.module, key_arr, name=gv.name + '.key')
        key_gv.global_constant = True
        key_gv.linkage = 'internal'
        key_gv.initializer = ir.Constant(key_arr, bytearray([key]))
```

Before encryption (in IR):

```
@".jk_str.0" = internal constant [19 x i8] c"Hello from JOCKY!\00"
```

After encryption (in IR):

```
@".jk_str.0"      = internal constant [19 x i8] c"\02'\x26\x26%J..."
@".jk_str.0.key"  = internal constant [1 x i8]  c"J"
```

An AV string scanner finds:

- No recognisable English words
- No recognisable system strings (no `svchost.exe`, `malware.exe`, `procs_list`, etc.)
- Random-looking bytes at every offset

The key is different per string per build

- Different build → different key for every string → different encrypted bytes
- This contributes to polymorphism independently of Pass 1

Why not a stronger cipher?

For the SIH demo, XOR is sufficient to demonstrate the concept. A production system might use AES or ChaCha20. However, stronger ciphers:

1. Require a runtime key schedule (more complex decryptor stub)
2. May themselves be detected as encryption routines by heuristic AV

XOR is undetectable as a cipher unless the AV knows the JOCKY binary format specifically.

The decryptor stub (future work)

The current implementation does not include a runtime decryptor. For the native binary to work with obfuscation AND readable strings, the compiled executable would need to:

1. At startup (before `start()` runs), iterate over all `.jk_str.*` globals
2. Load the corresponding `.key` value
3. XOR-decrypt each byte in-place
4. Then call `start()` normally

This would be implemented as an LLVM IR function (`_jocky_init`) called before `start()`, using `llvm.run_static_constructors()` — the same mechanism C++ uses for global constructors. This is marked as future work for SIH scope.

Pass 3 — Entropy Noise

Goal

Further differentiate binaries by adding another random constant that changes the byte pattern at a different position in the binary than Pass 1's 16-byte marker.

Implementation

```
def _pass_instruction_substitution(self) -> None:
    noise = ir.GlobalVariable(self.module, ir.IntType(64), name='_jocky_entropy')
    noise.global_constant = True
    noise.linkage = 'internal'
    noise.initializer = ir.Constant(ir.IntType(64), random.randint(0, 2**63 - 1))
```

In the IR:

```
@_jocky_entropy = internal constant i64 7284619203847561829
```

This is a 64-bit (8-byte) random value. Combined with the 16-byte build ID, every binary has 24 bytes of unique random content across two different locations in the data section.

Demonstrating Polymorphism for SIH Evaluators

Run the compiler twice without `--run` (binary output mode):

```
python compiler.py tests/hello.jk
# Output includes: SHA-256: 1e3b5024...

python compiler.py tests/hello.jk
# Output includes: SHA-256: 1304a8d2... (different!)
```

The hashes are always different. This can be demonstrated live in front of evaluators. You can also show the `.ll` IR file (with `--emit-ir --no-obfuscate` for one, `--emit-ir` for another) side by side to show:

- Without obfuscation: plaintext strings visible in the IR
- With obfuscation: encrypted bytes, random build-ID, random entropy value

What to say to evaluators

"JOCKY demonstrates three layers of AV evasion:

Layer 1: Unknown syntax. The `.jk` source files use a custom grammar. No existing AV parser can understand them. AV engines that parse scripts (Python, PowerShell) simply fail on JOCKY code.

Layer 2: No readable strings. When compiled to a binary, string literals are XOR-encrypted with a fresh random key. A static string scanner finds no readable text — no function names, no system paths, no identifying content.

Layer 3: Polymorphic binaries. Every compilation of the same source produces a binary with a completely different SHA-256 hash. Hash-based AV signatures become useless — there is no single hash to blacklist."

Security Limitations (Honest Assessment)

1. **Runtime decryptor not implemented.** The native binary currently produces garbled output when obfuscation is on. Compile with `--no-obfuscate` for functional native binaries.
2. **The encryption key is stored alongside the data.** The `.key` globals are in the same binary. A dedicated analyst can find the key and decrypt the strings. This defeats forensic analysis, not targeted reverse engineering.
3. **The obfuscation pass structure itself could be fingerprinted.** An AV vendor who reverse-engineers a JOCKY binary could identify the `_jocky_build_id` and `_jocky_entropy` globals as obfuscation markers and write a JOCKY-specific signature. Production use would rename or restructure these.
4. **LLVM IR is deterministic except for the random globals.** If the three obfuscation globals are identified and stripped, the remaining bytes are identical across builds.

These are acceptable limitations for an SIH hackathon demonstration.